Mappeoppgave: Tur- og aktivitetsregister

IDATx1003 Programmering 1, høsten 2026

v2.0

Sammendrag

Mappeoppgave for emnet IDATx1003 Programmering 1 for studieprogrammet Bachelor i ingeniørfag, data (dataingeniør) ved NTNU Trondheim, Gjøvik og Ålesund.

Revisjonshistorikk

Versjon	Dato	Forfatter	Beskrivelse
1.0	2026-09-25	Muhammad Ali Norozi m.fl.[*]	Første utgave
2.0	2026-10-20	Muhammad Ali Norozi m.fl.[*]	Utvidet dokumentet med Del 2 (av 3) i kapittel 5
3.0	2026-10-30	Muhammad Ali Norozi m.fl.[*]	Utvidet dokumentet med Del 3 (av 3) i kapittel 6

([*] Muhammad Ali Norozi, Arne Styve)

NTNU

Kunnskap for en bedre verden

Innhold

1 Om dokumentet **4**

2 Om mappevurdering i IDATx1003 Programmering 1 **5**

3 Prosjektoppgaven - Tur- og aktivitetsregister **6**

3.1 Nivåer . . . 6
3.2 Prosjektrapport . . . 6
3.3 Kravspesifikasjon . . . 6
3.3.1 Funksjonelle krav for Nivå 1 . . . 7
3.3.2 Funksjonelle krav for Nivå 2 . . . 7
3.3.3 Egne utvidelser . . . 8
3.4 Avgrensninger . . . 8
3.5 Bruk av KI i prosjektet . . . 8
3.5.1 Egen læring og kreativitet først . . . 8
3.6 KI-assistenter i ulike roller . . . 9
3.7 Krav til dokumentasjon: KI-logg . . . 9
3.7.1 Eksempler på KI-verktøy . . . 10
3.7.2 KI-erklæring (NTNU) . . . 10
3.8 Øvrige krav til prosjektet (ikke-funksjonelle krav) . . . 11
3.8.1 Kodekvalitet . . . 11
3.8.2 Enhetstesting . . . 11
3.8.3 Unntakshåndtering . . . 11
3.8.4 Versjonskontroll/Kildekodekontroll . . . 11
3.8.5 Java-biblioteker og Maven-plugins (valgfritt i Trondheim) . . . 11

4 Mappe del 1 (av 3) **15**

4.1 Kodeprosjekt . . . 15
4.2 Rapport . . . 16
4.3 Leveranse til veiledningsmøte . . . 16

5 Mappe Del 2 av 3 **18**

5.1 Nivå 1 - Minimal Viable Product (MVP) . . . 18
5.1.1 Enhetstest av registerklassen for aktiviteter . . . 18
5.1.2 Oppdatere applikasjonsklassen . . . 18
5.2 Nivå 2: Utvidet funksjonalitet – Register for loggførere, grupper og deling . . . 19
5.2.1 Enhetstest av de nye klassene . . . 19
5.2.2 Oppdatere applikasjonsklassen . . . 19

6 Mappe del 3 av 3 **22**
6.1 Hvilke læringsmål vi ønsker å teste i del 3 . . . 22
6.2 Krav til del 3 . . . 23
6.2.1 Krav til Programmet/koden . . . 23
6.2.2 Git og GitHub . . . 23
6.3 Krav til rapport . . . 23
6.3.1 Noen kommentarer til rapportmalen . . . 23
6.4 Bruk av AI-verktøy som GitHub CoPilot og ChatGPT . . . 24

7 Viktige sjekkpunkter **25**
7.1 Nyttige referanser . . . 25

A Vedlegg **27**
A.1 Nyttige verktøy . . . 27
A.1.1 UML-modellering . . . 27

B KI-deklarasjon **27**

Kodelister

1 Eksempel på en pom.xml-fil med riktige versjoner - gjelder for de som skal bruke Maven prosjekt . . . 13

1 Om dokumentet

Dette dokumentet inneholder 3 deler:

Om mappevurdering - her er det en detaljert beskrivelse av hvordan mappevurdering gjennomføres i emnet, hvilke deler mappen består av, og hvordan mappen skal leveres

Kravspesifikasjon - Beskrivelse av prosjektet i sin helhet

Mappen Del 1 - Hva som skal gjøres i del 1 av 3 - NB! LES DENNE GRUNDIG FØR DU STARTER MED KODINGEN!!

2 Om mappevurdering i IDATx1003 Programmering 1

I dette emnet er det ingen skriftlig eller muntlig eksamen. Istedenfor benyttes mappe som vurderingsform. Mappen skal løses **individuelt**. Mappen består av et programmeringsprosjekt samt en rapport.

Mappen leveres ut rundt uke 40 og går parallelt med øvrige læringsaktiviteter ut semesteret. Detaljert prosjektbeskrivelse blir publisert i 3 deler:

- Del 1 - cirka uke 40: Fokuserer på design av entity-klasse(r) samt versjonskontroll og enhetstesting.
- Del 2 - cirka uke 43: Fokuserer på samling(er) og forretningslogikk.
- Del 3 - cirka uke 45: Fokuserer på ferdigstillelse av applikasjon med et tekstbasert brukergrensesnitt og komplette enhetstester. I tillegg skal dere utvide løsningen med egne forslag/ideer.

Hver ny del bygger på foregående del. Når en ny del publiseres, vil dere få muntlig tilbakemelding på arbeidet som er gjort så langt, med mulighet til å justere/endre/forbedre løsningen. Dere vil få i alt 3 slike tilbakemeldinger, der den siste tilbakemeldingen gis i uke 47.

Endelig fungerende løsning leveres i GitHub og som ZIP-fil sammen med en rapport i Inspera innen fristen (se Blackboard for detaljer om innlevering).

Vekting mellom rapport og prosjekt mot endelig karakter:

- Rapport - 30%
- Prosjekt - 70%

Kilder:

- Hva forskriften sier om mappe som vurderingsform:
  `https://lovdata.no/dokument/SF/forskrift/2015-12-08-1449/KAPITTEL_5#%C2%A75-11`

3 Prosjektoppgaven - Tur- og aktivitetsregister

NB! Her følger en beskrivelse av den endelige løsningen som skal leveres til slutt i prosjektet. Senere i dokumentet er det en detaljert beskrivelse av hva dere skal utvikle i hver av de 3 delene. Ikke start på utviklingen/kodingen uten å lese hva som kreves i hver del!

Det skal utvikles en tur- og aktivitetsregister-løsning der brukeren kan loggføre turer og aktiviteter, for eksempel fjellturer, sykkelturer eller byvandringer. Registeret kan gjerne være spisset mot et tema, som for eksempel *fjellturer*, *sykkelturer* eller *byvandringer*. Dette velger du selv.

For å støtte oppstarten av prosjektet gis det eksempler på et *fjellturregister*.

Applikasjonen skal være en enkeltstående applikasjon som kjører på **en** datamaskin (altså ingen nettverk). Derfor må navnet til den som loggfører aktiviteten legges inn for hver aktivitet, og man kan legge inn ulike personer på de ulike aktivitetene.

Flere personer kan bruke samme applikasjon lokalt. I nivå 2 skal det også være mulig å **dele** informasjon om en aktivitet med andre loggførere i systemet – enten utvalgte personer eller en gruppe. Delingen skjer **lokalt i samme applikasjon** (ikke via nettverk, e-post eller lignende).

3.1 Nivåer

Ønsket funksjonalitet i applikasjonen er inndelt i 2 nivå:

- **Nivå 1: Et «Minimum Viable Product»-nivå (MVP)**. Består av helt grunnleggende funksjonalitet som det er forventet at alle skal kunne klare å implementere.
- **Nivå 2: Utvidet funksjonalitet.** Her vil det gis noen utfordringer (fortsatt innenfor pensum) som kanskje ikke alle vil klare å implementere.

For å oppnå full score (og høyeste karakter) forventes at Nivå 2 er implementert. Kravene til nivå 1 og 2 følger i kapittelet «Kravspesifikasjon» under.

3.2 Prosjektrapport

Det skal skrives en rapport for prosjektet. I rapporten skal dere forklare hvordan løsningen er bygget opp, hvilke valg som er tatt underveis, hvordan dere har anvendt designprinsipper og så videre.

Rapporten skal være på **ca. 2500–3000 ord**. I tillegg kommer figurer og eventuelt små kodesnutter for å vise hvordan utvalgte problemer er løst. Rapportens språk er enten norsk eller engelsk.

Det er lurt å starte på rapporten allerede fra starten. Dokumentmal er tilgjengelig på Canvas.

3.3 Kravspesifikasjon

En kravspesifikasjon består gjerne av to hovedkategorier av krav:

- **Funksjonelle krav** - dette er krav til brukerfunksjonalitet, altså hva bruker av applikasjonen skal være i stand til å utføre med løsningen.
- **Ikke-funksjonelle krav** - dette er øvrige krav som for eksempel *brukervennlighet*, krav til *teknologier* som skal benyttes, krav til *responstid*, *robusthet* og så videre.

Følgende funksjonelle krav gjelder, fordelt på nivåer.

3.3.1 Funksjonelle krav for Nivå 1

Her forventes det at løsningen inneholder kun **en** samling (register) av aktiviteter.

En bruker skal:

- kunne opprette en aktivitet i registeret. Aktiviteten skal inneholde informasjon som:
  - Navnet til den som loggfører aktiviteten
  - En tittel på aktiviteten, for eksempel "Tur til Preikestolen"
  - En lengre tekst (beskrivelse av turen/aktiviteten)
  - Dato og klokkeslett for når aktiviteten ble registrert
- kunne skrive ut alle aktivitetene på en strukturert og oversiktlig måte
- kunne søke etter aktiviteter på en bestemt dato
- kunne slette aktiviteter

3.3.2 Funksjonelle krav for Nivå 2

Her forventes at det lages et eget register over **loggførere** (personer som registrerer aktiviteter). Hver gang en ny aktivitet logges, kan for eksempel brukeren få opp en liste over tidligere loggførere der bruker kan velge en av disse, eller legge til en ny loggfører.

I tillegg skal løsningen støtte **grupper** og **deling** av aktiviteter mellom loggførere i samme applikasjon.

En bruker skal:

- kunne velge blant tidligere loggførere når ny aktivitet opprettes
- kunne opprette og vedlikeholde grupper av loggførere (f.eks. "Fjellgjengen" eller "Sykkelklubben")
  - en gruppe skal ha et navn
  - en gruppe skal kunne inneholde én eller flere loggførere
  - det skal være mulig å legge til og fjerne loggførere fra en gruppe
- kunne dele en aktivitet med:
  - én eller flere utvalgte loggførere, **eller**
  - en gruppe (slik at alle medlemmer i gruppen får tilgang til aktiviteten)
- kunne angi om en aktivitet er **privat** (kun synlig for den som loggførte den) eller **delt**
- kunne vise aktiviteter som er delt med en bestemt loggfører
- kunne søke etter aktiviteter i et bestemt tidsrom (fra-dato, til-dato) og vise aktivitetene
- kunne søke etter aktiviteter fra en bestemt loggfører og vise aktivitetene
- kunne søke etter aktiviteter basert på et ord i tittel eller beskrivelse og vise aktivitetene
- kunne hente ut statistikk i form av antall aktiviteter pr loggfører. Kan for eksempel vises som en tabell der kolonne 1 er en liste av loggførere med antall aktiviteter pr loggfører i kolonne 2.

Viktig avklaring om deling: Deling betyr at informasjon om aktiviteten gjøres synlig for andre loggførere *inne i samme lokale applikasjon*. Det skal **ikke** implementeres nettverkskommunikasjon, filoverføring mellom maskiner, e-post eller lignende.

3.3.3 Egne utvidelser

I tillegg til de definerte funksjonelle kravene kan man utvide eller endre løsningen etter egne ideer og ønsker. Egne utvidelser vil telle positivt i vurderingen av besvarelsen.

Eksempler på egne utvidelser (valgfritt):

- type aktivitet (f.eks. fjelltur, sykkeltur, byvandring)
- varighet eller distanse
- vurderingskarakter (1–5) på turen
- sted/lokasjon
- fjerne deling av en aktivitet (gjøre den privat igjen)
- vise antall delte vs. private aktiviteter i statistikken

3.4 Avgrensninger

Følgende avgrensninger av oppgaven gjelder:

- Applikasjonen skal være tekstbasert (ikke grafisk brukergrensesnitt).
- Applikasjonen skal kunne kjøres fra IDE og fra kommandolinje
- Applikasjonen skal være enkeltstående og kjøre på **en** datamaskin (ingen nettverk)
- Deling av aktiviteter skjer kun lokalt mellom loggførere/grupper i samme applikasjon
- Programmeringsprosjektet **skal** være satt opp med **Maven** som byggsystem (Maven er valgfritt i Trondheim)

3.5 Bruk av KI i prosjektet

Det er tillatt, og **forventet** at du bruker en (eller flere) KI-assistent(er) aktivt i dette prosjektet, men på **en reflektert og kritisk måte**. Målet er *ikke* å unngå KI, og heller *ikke* å la KI løse oppgaven for deg – målet er at du skal trene på å bruke KI som et verktøy for læring og kvalitetssikring, samtidig som du beholder eierskap til og forståelse av egen kode.

Bruken av KI skal være i tråd med NTNUs retningslinjer for bruk av kunstig intelligens i studentoppgaver. Hovedprinsippet er at KI skal være et **hjelpemiddel**, mens besvarelsen (kode, designvalg og rapport) skal utarbeides av **dere selv**.

Se også NTNUs kunnskapsbase: Kunstig intelligens i studentoppgaver, samt se NTNU sine sider om KI: https://ntnu.no/ki.

3.5.1 Egen læring og kreativitet først

Målet med mappeprosjektet er at dere skal lære å programmere, modellere et problem og ta egne designvalg. Derfor gjelder følgende:

- Bruk egen kreativitet når dere velger tema (f.eks. type turer/aktiviteter), navngir klasser, utformer menystruktur og løser problemer.
- KI-verktøyet skal brukes som sparringpartner/rådgiver og oppslagsverk – **ikke** for å generere ferdig kode som kopieres direkte inn i prosjektet.
- Ikke be KI om "skriv hele løsningen" eller "implementer nivå 1 og 2 for meg". Be heller om forklaringer, hint, alternativer eller hjelp til å forstå feilmeldinger.
- Eventuell kode som foreslås av KI skal gjennomgås kritisk, forstås og bearbeides før den eventuelt tas i bruk.

- Dere skal kunne forklare all kode dere leverer: hva den gjør, hvorfor den er skrevet slik, og hvilke valg som er tatt.
- Kommenter i koden der dere har fått vesentlig hjelp av KI-verktøy.

Å levere kode dere ikke forstår, eller som i stor grad er generert uten egen bearbeiding, svekker læringsutbyttet og kan være i strid med kravene til akademisk redelighet.

3.6 KI-assistenter i ulike roller

KI kan brukes i tre ulike roller i dette prosjektet:

1. **KI som veileder/lærer** – for å forklare konsepter, Java-syntaks, feilmeldinger eller eksempler du er usikker på.
2. **KI som kodegjennomgår («reviewer»)** – for å få kritikk av *din egen ferdige* kode opp mot krav til robusthet, validering, navngiving og testdekning. KI skal *ikke* skrive om koden for deg – be den heller peke på svakheter og forslag til forbedring.
3. **KI som «adversarial tester»** – for å generere forslag til edge cases og grensetilfeller du selv må vurdere og bestemme om skal håndteres. Igjen skal ikke KI generere kode for deg, men indikere edge-cases som du må sørge for at blir testet.

For å få en KI-assistent til å oppføre seg i en gitt rolle, må du gi assistenten en innledende instruksjon før du begynner å «prate» med den. Her er eksempel på en slik innledende «System promp» for å instruere KI-assistenten til å innta rollen som veileder:

```
You are an explanation assistant.
You help users understand general coding principles, theories, and concepts.
You may explain coding topics in detail.
You must never give code or pseudo code to the user.
You may never use the code that the user offers directly.
You may never perform coding tasks for the user.
You clarify their understanding with examples or metaphors that illustrate the concept.
You may never solve any coding tasks or provide solutions.
You encourage users to ask questions that help them understand broader concepts
rather than individual tasks.
```

3.7 Krav til dokumentasjon: KI-logg

I rapporten skal du inkludere en **KI-logg** som et vedlegg, i tabellform, med minst én rad per fase/del (gjerne flere). Tabellen skal inneholde:

Fase	Formål/Rolle	Prompt brukt (kort)	Sentralt forslag fra KI	Akseptert?	Begrunnelse
1	Reviewer	«Review this class for validation gaps»	Foreslo sjekk av negativ kapasitet	Ja	Manglet – lagt til sjekk og test
2	Tester	«Suggest edge cases for gate assignment»	Foreslo å sjekke samtidig gate-bytte	Nei	Utenfor omfang – dokumentert som kjent begrensning

Krav: Du bør ha **minst ett dokumentert tilfelle per fase hvor du er uenig med, korrigerer, eller avviser** et forslag fra KI – enten fordi forslaget var feil, fordi det lå utenfor pensum (f.eks. forslag om arv eller egne exception-klasser), eller fordi det var unødvendig komplisert for et førsteårsprosjekt. Dette er den viktigste dokumentasjonen av at du har brukt KI kritisk, og ikke bare kopiert forslag ukritisk.

Muntlig oppfølging: I veiledningsmøtene må du være forberedt på å forklare *enhver* del av din egen kode – inkludert kode der KI har bidratt med forslag. Dette gjelder ikke for å avsløre juks, men fordi forståelse – ikke bare korrekthet – er det som til syvende og sist vurderes.

3.7.1 Eksempler på KI-verktøy

GitHub Copilot: Bruk chatbot'en, og ikke «code completion». Chatboten er inkludert i plugin for IntelliJ og VSCode. GitHub Copilot er mye bedre enn ChatGPT i forhold til programmeringskode. Studenter får gratis tilgang til GitHub CoPilot ved å søke om «educational license» hos GitHub (https://education.github.com/pack)

Microsoft Copilot: Tilsvarer ChatGPT. Er det KI-verktøyet som formelt er godkjent av NTNU. Finnes ikke som plugin til IntelliJ.

Claude Code: Denne finnes ikke som gratisversjon eller med studentlisens. Selv om Claude oppfattes som «den beste» på koding, er de andre verktøyene mer enn gode nok for det nivået dere skal kode på i dette prosjektet. Ingen grunn til å betale dyrt for Claude for dette prosjektet.

NTNU og UiO's egen GPT https://gpt.ntnu.no: Dete er et trygt alternativ da samtlige språkmodeller kjører enten på NTNU sine egne servere elelr på servere i Europa. Språkmodellem **Kimi K2.6** er god på kode og kan for eksempel greit brukes til å kontrollere kvalitet og robusthet i koden din. Dette verktøyet kan ikke integreres i IntelliJ/VSCode som plugin.

3.7.2 KI-erklæring (NTNU)

Dersom dere bruker KI-verktøy, skal dette gjøres rede for. Ved NTNU innebærer det typisk:

- en **KI-deklarasjon** / redegjørelse i rapporten (eget avsnitt), og/eller
- utfylling av eventuelt **KI-deklarasjonsskjema** dersom det kreves ved innlevering (f.eks. i Inspera) for emnet.

Hensikten er både å bevisstgjøre dere om egen bruk, og å informere sensor om hvordan KI er benyttet. Deklarasjonen skal beskrive *hvilke* verktøy som er brukt, *til hva*, og gjerne med korte eksempler på prompts og hvordan svarene ble brukt/bearbeidet.

Det er greit å bruke **forenklet KI-deklarasjonsskjema** for dette prosjektet.

Eksempler på hva som bør oppgis:

- hjelp til å forstå Java-konsepter (f.eks. `LocalDateTime`, unntak, lister)
- hjelp til feilsøking / tolking av feilmeldinger
- språkvask eller strukturforslag til rapporten
- forslag til testdekning (positive/negative tester) som dere deretter implementerer selv

Husk: språkmodeller er **ikke** pålitelige faktakilder. Kontroller alltid svar mot pensum, JavaDoc og egen testing. Last ikke opp personopplysninger eller sensitiv informasjon i KI-verktøy.

Dere står selv ansvarlig for det faglige innholdet i både kode og rapport.

3.8 Øvrige krav til prosjektet (ikke-funksjonelle krav)

Følgende krav og betingelser gjelder for hele mappen:

3.8.1 Kodekvalitet

Det forventes høy kodekvalitet i prosjektet. Bruk verktøy som **CheckStyle** (med Google sine regler) og **SonarQube**. Begge verktøy finnes som plugins til IntelliJ og VSCode med flere (se vår emnewiki for detaljer).

3.8.2 Enhetstesting

Det skal skrives enhetstester for den delen av koden som er forretnings-kritisk (forretnings-logikken inklusive entitetsklassene), altså for den koden som er viktigst for å oppfylle sentrale krav. Feil her vil få store negative konsekvenser for programmet. Negativ testing er også viktig. Det er ikke fornuftig å lage enhetstester til klasser som utgjør brukergrensesnittet, med mindre det finnes metoder som kan testes uten at en bruker er involvert.

3.8.3 Unntakshåndtering

Uønskede hendelser og tilstander som forstyrrer normal flyt skal håndteres på en god måte. Håndteringen skal være rimelig og balansert (ikke for mye, ikke for lite) med det formål å gjøre koden mer robust. Vi gjør oppmerksom på at unntakshåndtering ikke er eksplisitt angitt i oppgavene under. Dere må selv avgjøre når og hvordan unntak skal implementeres. En viktig sammenheng der unntak bør brukes er validering av parametre til metoder. Merk at det er ikke pensum å lage egne unntaksklasser, dette kommer i Programmering 2. Her er det snakk om å bruke de eksisterende unntaksklassene, og velge fornuftig blant disse. M.a.o. ikke bruk **IllegalArgumentException** over alt.

3.8.4 Versjonskontroll/Kildekodekontroll

Prosjektet skal legges under versjonskontroll lokalt samt på GitHub. Krav til versjonskontroll er ytterligere beskrevet i del 1.

3.8.5 Java-biblioteker og Maven-plugins (valgfritt i Trondheim)

Følgende versjoner av Java JDK, biblioteker og Maven samt plugins skal benyttes (versjoner som var tilgjengelige pr august 2026):

- **Java:** Siste tilgjengelige LTS (Long Time Support) versjon **skal** benyttes: versjon 25 (som var gjeldende ved semesterstart)
- **Maven**: versjon 3.9.16
- **MavenPlugins** (<build><plugins> i pom.xml) - er valg fritt i Trondheim
  - *org.apache.maven.plugins:maven-compiler-plugin:3.15.0*
  - *org.apache.maven.plugins:maven-surefire-plugin:3.6.0*
- **Bibliotek** (<dependencies> i pom.xml)
  - **JUnit6**: *org.junit.jupiter:junit-jupiter-engine:6.1.3*

For andre biblioteker dere måtte velge/trenge, husk å **alltid** spesifisere eksakt versjon, **aldri** bruke RELEASE eller LATEST!

Kodeliste 1: Eksempel på en pom.xml-fil med riktige versjoner - gjelder for de som skal bruke Maven prosjekt

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <!-- TODO: Replace with your own groupID and artifactID -->
    <groupId>edu.ntnu.iir.bidata</groupId>
    <artifactId>your-artifact-id</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <!-- Per August 2026 the LTS version is 25 -->
        <maven.compiler.release>25</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>6.1.3</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <!-- The Java compiler -->
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.15.0</version>
            </plugin>

            <!-- The unit-test engine Surefire -->
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.6.0</version>
            </plugin>
        </plugins>
    </build>
</project>
```

Del 1 (av 3)

Prosjektoppsett og grunnleggende klasser.

4 Mappe del 1 (av 3)

I første del av mappen er fokuset å:

- forstå oppgaven/prosjektet
- få opprettet prosjektet som et Maven-prosjekt (ikke nødvendig for Trondheim)
- sette prosjektet under versjonskontroll koblet til GitHub (git.ntnu.no for Trondheim)
- implementere entitetsklassen (klassen som representerer en aktivitet/tur, "Activty" class)
- implementere testklasse for å teste entitetsklassen med tilsvarende navn.
- starte på rapporten

Detaljene følger under.

4.1 Kodeprosjekt

Dere i **Ålesund** og **Gjøvik** får tilgang til et ferdig oppsatt prosjekt via **Classroom50** (tidligere GitHub Classroom) for mappen (**blir annonsert på Canvas**). Prosjektet er satt opp som et **Maven**-prosjekt. For å få tilgang til dette prosjektet og få opprettet sitt eget repository for Mappen på GitHub, se Canvas og siden "Opprette Mappe-prosjektet fra GitHub" under modulen "Mappevurdering" i Canvas.

Dere i **Trondheim** får tilgang og lenk til template prosjekt via **NTNU GitHub** for mappen også annonsert på Canvas. Detaljert instruks gis på README.md. Dere skal opprett eget respository fra templaten, se Canvas og siden "Opprette Mappe-prosjektet fra GitHub" under modulen "Mappevurdering" i Canvas.

Når prosjektet er åpnet og klonet fra Git, last inn og åpne prosjektet i et utviklingsmiljø (IntelliJ, VSCode eller lignende). Følg videre instrukser under (NB! Dere kan ikke bruke BlueJ på dette prosjektet):

- Lage entitetsklasse for en aktivitet/tur (engelsk: "Activity")
- Dokumenter klassen grundig (i henhold til standarden for Javadoc), inkludert
  - Rolle/ansvar
  - Hvilke informasjon klassen holder på og hvilke datatyper som er valgt for hver informasjon og hvorfor (begrunnelse). Typisk informasjon for nivå 1 er: navn på loggfører, tittel, beskrivelse, samt dato og klokkeslett for registrering
  - En vurdering av hvilke informasjon skal kun settes ved opprettelse av instans, og hvilken informasjon må kunne endres etter at instansen er opprettet
  - Hvordan klassen responderer på ugyldige data - hvilken strategi følger klassen (kaster unntak? setter dummy-verdi?)
- Implementer nødvendige felt i klassen, med egnede datatyper. Se eget avsnitt under for tips til håndtering av dato og tid.
- Lag enhetstester for å teste at klassen for aktivitet fungerer som den skal og er robust. Husk både *positive*- og *negative* tester.
- Kjør CheckStyle på koden din med Google-reglene. (Se wiki-siden for IDATx1003 for hvordan installere CheckStyle for IntelliJ)
- Opprett en klasse som på sikt skal bli brukergrensesnittet (UI) og dermed ta seg av all brukerinteraksjon. Opprett to metoder; start() og init().
- Bruk start-metoden til å skrive enkel testkode for å teste at aktivitet-klassen fungerer i henhold til kravspesifikasjonen ved for eksempel å opprette 3-4 instanser av klassen (f.eks. ulike fjellturer) og skrive ut objektene. Det kan lønne seg å lage en egen metode for å skrive ut detaljene til en aktivitet.
- Opprett til slutt en klasse som skal være selve applikasjonen, og som inneholder ***public static***

void main(String[] args)-metoden.
- Opprett main()-metoden. La denne metoden lage en instans av UI-klassen og kall deretter metodene init() og start() til UI-objektet.

Husk å utføre en commit (innsjekk) til lokalt Git-repository med jevne mellomrom. Husk å alltid legge til en kommentar til innsjekken. Husk også å utføre push til GitHub med jevne mellomrom for å oppbevare koden trygt i skyen. Dette er svært viktig i tilfelle man får problemer med datamaskinen.

Håndtere dato og tid

Det finnes flere måter å håndtere dato og tid på i Java. For JDK v25 er den anbefalte måten for både å håndtere dato og tid i en og samme datatype, å bruke klassen **LocalDateTime** (klikk på lenken for å se JavaDoc for klassen).

Gjør dere godt kjent med denne klassen ved å lese dokumentasjonen, se på eksempelkode, lage enkel testkode, gjerne i CodePad'en i BlueJ, og spør også gjerne KI (men vær kritisk til svarene).

4.2 Rapport

Begynn på rapporten allerede nå:

- Sett dere inn i rapportmalen
- Fyll ut forside, og skriv innledningen
- Lag et UseCase-diagram som viser hvilken funksjonalitet løsningen skal tilby en bruker.
- Start på kapitlene om teori og metode.
- I resultat-kapitelet kan dere allerede nå gi en kort beskrivelse av hvilke klasser dere ser for dere å lage i prosjektet, og vise et klassediagram som viser avhengigheten mellom dem.

4.3 Leveranse til veiledningsmøte

Til veiledningsmøtet for del 1 forventes at følgende presenteres:

- Prosjektkoden som viser implementerte klasser iht spesifikasjonen for Del 1.
- Implementerte enhetstester (positive og negative) for samtlige klasser (med unntak av UI-klassen)
- Påbegynt rapport der minimum følgende er på plass:
  - Innledning
  - Kravspesifikasjon
  - Påbegynt KI-logg som dokumenterer hvordan KI har vært benyttet

Del 2 (av 3)

Register over aktiviteter

5 Mappe Del 2 av 3

I denne delen av mappen skal du implementere registeret som skal holde på alle aktivitetene i (nivå 1) og register over loggførere, samt støtte for grupper og deling (nivå 2), med tilhørende enhetstest-klasse(r) som tester registerklassen(e) både med negative og positive tester.

5.1 Nivå 1 - Minimal Viable Product (MVP)

Implementer en klasse som er ansvarlig for å holde på en samling av aktiviteter med tilhørende funksjonalitet. Du skal selv:

- Velge navn på klassen som skal representere registeret.
- Vurdere hvilken klasse fra Java SDK du tenker er passende å bruke for å lagre alle aktivitetene i (ArrayList, HashSet, HashMap og så videre). Husk å begrunn i rapporten hvorfor du valgte nettopp denne klassen fra SDK'en.

Fra listen over funksjonelle krav fra kravspesifikasjonen (3.3.1), bør register-klassen som minimum ha støtte for følgende funksjonalitet:

- En metode for å legge til en aktivitet.
- En metode for å søke opp aktiviteter på en gitt dato. Her må du også vurdere om du skal returnere den første aktiviteten, eller alle aktiviteter på en bestemt dato.
- En metode for å slette en aktivitet.
- En metode som gir tilgang til samlingen av aktiviteter, slik at klassen som representerer brukergrensesnittet har mulighet til for eksempel å skrive ut alle aktivitetene. Aktivitetene bør da være sortert enten i stigende eller synkende rekkefølge basert på dato/tid.

I tillegg til disse metodene bør du selv tenke ut og implementere øvrige metoder som kan være nyttig i et slikt register.

NB! Beskriv i rapporten løsningen du valgte for å returnere tilgang til en sortert liste av aktiviteter. Beskriv hvordan du kom frem til løsningen (søkte på nett, ChatGPT, GitHub CoPilot, spurte en venn og så videre), og forklar med egne ord koden du har implementert (dette for å vise at du forstår koden du har implementert).

Dersom kode er inspirert av en løsning fra andre (for eksempel nettside) er det viktig at kilde henvises til og valg begrunnes.

5.1.1 Enhetstest av registerklassen for aktiviteter

Opprett enhets-tester for å teste registerklassen. Husk at det er smart å planlegge på forhånd hva du skal teste i registerklassen og hvordan, og spesielt hvordan du skal teste for å få utført negativ testing. Dette er det lurt å skrive i JavaDoc'en til testklassen FØR du begynner å kode testene.

Husk både positiv(e) og negativ(e) tester. Kontroller gjerne dekningsgrad (test-coverage) for å sjekke at mest mulig av koden i register-klassen dekkes av testene. Bare husk at 100% dekning ikke er ensbetydende med at testene dine er gode.

5.1.2 Oppdatere applikasjonsklassen

Oppdater init() og start() metodene i klassen som representerer applikasjonen (og på sikt vil representere det tekstbaserte brukergrensesnittet):

- **init()**: Her legger du inn all kode som er nødvendig for å initialisere applikasjonen ved oppstart, som f.eks. å opprette instansen av register-klassen.
- **start():** Oppdater denne metoden til å ta i bruk den nye register-klassen. Opprett 3-4 aktiviteter i kode som du legger til i registeret (f.eks. ulike fjellturer). Test deretter noe av funksjonaliteten til registeret. Du kan f.eks. allerede nå tenke på å opprette en metode som skriver ut oversikt over aktivitetene til konsollet basert på registrerte aktiviteter. Det er lurt å lage en egen metode for å skrive ut denne oversikten. Vent med å implementere full meny med input fra brukeren. Dette kommer i del 3.

5.2 Nivå 2: Utvidet funksjonalitet – Register for loggførere, grupper og deling

Det vil være svært sannsynlig at flere aktiviteter er loggført av samme person. I stedet for å lagre navnet til loggføreren som en tekst i et felt i aktiviteten, kan loggfører være representert med egen klasse (engelsk: *Person*). Når en aktivitet opprettes med en loggfører som ikke finnes fra før, bør det være mulig å opprette en ny loggfører og lagre vedkommende i eget register.

I tillegg skal nivå 2 støtte **grupper** og **deling** av aktiviteter mellom loggførere i samme lokale applikasjon (ingen nettverk):

- En klasse for gruppe (engelsk: *Group*) med navn og medlemmer (loggførere).
- Mulighet for å opprette grupper, samt legge til og fjerne loggførere fra en gruppe.
- Mulighet for å dele en aktivitet med én eller flere utvalgte loggførere, eller med en gruppe.
- Mulighet for å angi om en aktivitet er **privat** eller **delt**.
- Mulighet for å hente/vise aktiviteter som er delt med en bestemt loggfører.

Se kravspesifikasjon i 3.3.2 for innspill til hvilken funksjonalitet registrene bør ha for å imøtekomme de funksjonelle kravene. Dokumenter dine valg i rapporten.

Tips: Tenk gjennom hvordan en aktivitet skal referere til loggfører (objektreferanse, ikke bare tekst), og hvordan deling kan modelleres (f.eks. en liste/samling av loggførere og/eller grupper aktiviteten er delt med). Begrunn valgene i rapporten.

5.2.1 Enhetstest av de nye klassene

Opprett enhets-tester for å teste både klassen for *Person* (loggfører), eventuelle gruppe-/delingsrelaterte klasser, og tilhørende registerklasser. Husk at det er smart å planlegge på forhånd hva du skal teste og hvordan, og spesielt hvordan du skal teste for å få utført negativ testing. Dette er det lurt å skrive i JavaDoc'en til testklassen FØR du begynner å kode testene.

Husk både positiv(e) og negativ(e) tester. Kontroller gjerne dekningsgrad (test-coverage) for å sjekke at mest mulig av koden i register-klassene dekkes av testene. Bare husk at 100% dekning ikke er ensbetydende med at testene dine er gode.

5.2.2 Oppdatere applikasjonsklassen

Oppdater init() og start() metodene i klassen som representerer applikasjonen med funksjonalitet fra Nivå 2:

- **init()**: Legg til kode for å opprette registeret for loggførere, samt det som trengs for grupper/deling (avhengig av hvordan du modellerer dette).
- **start():** Oppdater denne metoden til å ta i bruk de nye klassene. Opprett 3-4 loggførere i kode som du legger til i registeret. Opprett gjerne også minst én gruppe med noen medlemmer.

Opprett deretter 3-4 aktiviteter med loggførere fra loggfører-registeret, der minst én aktivitet er privat og minst én er delt (med utvalgte personer og/eller en gruppe). Oppdater utskrift av en aktivitet nå når loggfører er en egen klasse, og vis også delingsstatus. Vent med å implementere full meny med input fra brukeren. Dette kommer i del 3.

Del 3 (av 3)

Ferdigstille applikasjon med brukergrensesnitt

6 Mappe del 3 av 3

Del 3 av mappen bygger videre på Del 1 og 2 for **Tur- og aktivitetsregister**.

Basert på tilbakemeldingen du har fått på del 2: gjennomfør nødvendig refaktorisering av koden din fra del 2.

Denne delen gjelder resten av mappe-perioden frem til innlevering i uke 50. Dere får tilbud om tilbakemelding på Mappen Del 3 i uke 47, etter dette er dere på egen hånd.

I denne siste delen av Mappeoppgaven skal du ferdigstille applikasjonen din og rapporten. I tillegg skal du nå sette ditt eget preg på applikasjonen:

- **Brukergrensesnitt:** Du skal implementere et brukergrensesnitt for tur- og aktivitetsregisteret. Du må gjerne tenke alternativt her. Er en meny beste løsning, eller er kommandoer fra kommandolinjen et bedre alternativ (tilsvarende «world-of-zuul»-prosjektet)? Grafisk brukergrensesnitt er pensum først i Programmering 2, så det gis ingen ekstrapoeng dersom du velger å implementere et grafisk brukergrensesnitt.
  Husk at her er det viktigere at du vektlegger brukervennlighet og et robust grensesnitt, enn et fancy brukergrensesnitt. Anbefales at du holder deg til pensum i emnet.
  Brukergrensesnittet skal gjøre det mulig å bruke funksjonaliteten fra kravspesifikasjonen (nivå 1, og nivå 2 dersom det er implementert), for eksempel å registrere aktiviteter, søke, slette, vise oversikt, samt (på nivå 2) håndtere loggførere, grupper og deling.
- Med utgangspunkt i opprinnelig kravspesifikasjon, hvilke endringer/forbedringer ville du ha gjort for at applikasjonen skal bli enda mer nyttig og brukervennlig for brukeren? Her har du lov til å avvike noe fra opprinnelig kravspesifikasjon og tilføre dine tanker og idéer (f.eks. type aktivitet, distanse/varighet, sted, vurderingskarakter, bedre oversikt over delte vs. private aktiviteter). Løsningen din må selvsagt fortsatt oppfylle de grunnleggende funksjonelle brukerkrav, men du står fritt til å endre på designet av klasser, valg av datatyper om du mener forslagene oppgitt i oppgaveteksten ikke er gunstige, og legge til ny funksjonalitet.
- Med utgangspunkt i designprinsippene **coupling** og **cohesion**, hvilke eventuelle endringer vil du gjøre i ditt design? Kan det være aktuelt å introdusere flere klasser for å oppfylle cohesion-prinsippet bedre, eventuelt innføre flere metoder (delegere)? Tenk spesielt på hvordan ansvar fordeles mellom aktivitet, loggfører, gruppe, registre og brukergrensesnitt.
- Hvordan møter din løsning prinsippet om lagdelt arkitektur?
- Beskriv i rapporten hva du har foreslått av endringer og hvordan du har valgt å implementere disse, og hvorfor disse endringene gir et bedre design i tråd med designprinsippene vi har lært i emnet.

Det er ikke krav om å lage enhetstester for klasser som håndterer brukerinteraksjon.

6.1 Hvilke læringsmål vi ønsker å teste i del 3

I tillegg til at du med mappen skal kunne demonstrere samtlige læringsmål i emnebeskrivelsen, vil vi i del 3 ha spesielt søkelys på:

- evnen til å levere et selvstendig arbeid
- evnen til å re-designe (refaktorere) egen løsning uten å tilføre nye feil
- evnen til å skrive fail-safe/robust kode
- evnen til å skrive kode som følger en etablert kodestil (Google), og som har god lesbarhet og er dokumentert i henhold til anbefalte industristandarder (JavaDoc)
- evnen til å implementere et robust design etter prinsipper som modularisering, coupling, cohesion, responsibility driven design

- brukervennlighet/god brukerinteraksjon

6.2 Krav til del 3

6.2.1 Krav til Programmet/koden

Følgende krav gjelder (fortsatt) til denne delen av oppgaven:

- Koden skal følge en bestemt kodestil (fortrinnsvis Google sin stil)
- Kodestilen skal verifiseres med CheckStyle-plugin (for IntelliJ, VSCode osv) og vise ingen/minimalt med regelbrudd ved levering.
- Klassene samt alle metoder og variabler (felt, parametere, lokale variabler) skal ha gode, beskrivende navn som tydelig gjenspeiler hvilken tjeneste en metode tilbyr, eller hvilken type verdi variablene representerer/holder på.
- Alle navn på klasser, metoder og variabler skal være på engelsk
- Koden skal være dokumentert (på engelsk) iht standarden for JavaDoc (se hvordan JDK'en er dokumentert, for inspirasjon og som referanse)

Du velger selv hvilken IDE (utviklingsverktøy) du vil bruke på prosjektet (IntelliJ, VS Code, Eclipse etc), men prosjektet skal være satt opp som et Maven-prosjekt.

6.2.2 Git og GitHub

Prosjektet ditt skal ha både et lokalt repository, og være koblet til et sentralt repository på GitHub.

Underveis i prosjektet skal du:

- Foreta hyppige commits til lokalt repository med gode commit-meldinger, samt utføre push til sentralt repository.
- Bruke tagger for å markere "release"-versjoner

Prosjektet skal til slutt pushes i sin helhet til sentralt repository (GitHub), i tillegg til å levers som ZIP-fil i Inspera.

6.3 Krav til rapport

Rapporten fra del 2 videreføres med tilbakemeldingene du fikk fra LA/Faglærer.

Rapporten skal være ca 2500 - 3000 ord!

Til samtalen for tilbakemelding på del 3:

- bør rapporten i størst mulig grad ferdigstilles. Kapitlene resultat og drøfting skal nærme seg ferdigstillelse, tilstrekkelig til at det er mulig å gi tilbakemelding.
- det er viktig at dere dokumenterer opprinnelige funksjoner/klasser contra refaktorerte funksjoner/klasser, og ikke minst hvorfor du valgte å refaktorere.

6.3.1 Noen kommentarer til rapportmalen

Rapportmalen dere har fått utlevert, er egentlig en mal som også benyttes til Bachelor oppgaver 3. året. Målet med å gi denne allerede i 1. semester, er at dere skal bli vant til å tenke **akademisk rapport** fra starten av, slik at det å skrive en slik rapport forhåpentligvis blir enklere for dere når dere kommer til Bachelor-oppgaven.

Alle akademiske/vitenskapelige rapporter følger omtrent samme mønster, så også denne malen:

1. **Innledning:** Her skriver dere kort om prosjektet som rapporten omhandler (tur- og aktivitetsregisteret).
2. **Bakgrunn – teoretisk grunnlag:** All utvikling er basert på teorier og bakgrunnsinformasjon (f.eks. fra andre publikasjoner eller kilder/bøker). Her skal dere kort beskrive de **teoriene** som dere har benyttet når dere har løst mappen, og henvise til **kildene** der dere har teoriene ifra (bøker, nettressurser osv). IKKE skriv om teorier her som der IKKE har benyttet, og ikke omtaler senere i rapporten!
3. **Metode-design:** Her beskriver dere kort hvilke **verktøy** dere har benyttet i prosjektet (IDE'er, osv), samt en kort beskrivelse av **arbeidsmetoden** dere har jobbet etter.
4. **Resultat:** Dette er det viktigste og største kapitlet i rapporten. Her skal dere beskrive den endelige løsningen dere har kommet frem til. Bruk UML diagrammer som use-case diagram(er), klassediagram(er), sekvensdiagram(er) osv. Beskriv kort hver klasse i forhold til ansvar/rolle (ikke gå i detalj på alle metoder og felt). Begrunn ditt designvalg med referanse til teoriene du har beskrevet i teoridelen. Ikke drøft løsningen din i dette kapittelet, det kommer i neste kapittel.
5. **Drøfting:** Nå er det tid for å reflektere over resultatene du har oppnådd i forhold til kravene som var gitt i oppgaven, og i forhold til teoriene om god design og robust løsning. Klarte du å lage en applikasjon som er designet med «high cohesion» og «low coupling»? Holder kodebasen din høy kvalitet og er veldokumentert? Beskriv og «bevis» ved å henvise til konkrete sider av løsningen din (f.eks. hvordan deling/grupper er modellert, eller hvordan UI er skilt fra forretningslogikk).
6. **Konklusjon:** En kort oppsummering av prosjektet som helhet. Maks en halv side.

Generelt:

- Pass på **den røde tråden** gjennom kapitlene; har du beskrevet en teori i teoridelen, så sørg for å henvis til denne i resultat- og/eller diskusjons-delen av rapporten. Teorier som ikke henvises til, skal ikke stå i rapporten.
- Terminologier osv: Dersom du *ikke* har benyttet deg av forkortelser/termer i rapporten, dropp kapittelet

6.4 Bruk av AI-verktøy som GitHub CoPilot og ChatGPT

Dersom du har benyttet deg av AI-verktøy som støtte/hjelp til å løse mappeoppgaven, skal dette dokumenteres tydelig både i koden (ved kommentarer) og i rapporten. Se også kravene til KI-bruk og KI-erklæring i hovedbeskrivelsen av prosjektet.

I rapporten skal du beskrive:

- Hvilke AI-verktøy du har benyttet (ChatGPT, GitHub CoPilot, Claude code, Cursor osv).
- Hva du har brukt AI-verktøy til (generere kode, stille spørsmål, hjelp til rapport osv)
- Der du har brukt AI-verktøy for hjelp til kode: dokumenter i rapporten de delene av koden som du har fått hjelp til av AI-verktøy, og beskriv hvorfor du mener foreslått kode løser problemet ditt, og hva koden gjør (du må forstå og kunne beskrive hva koden gjør).

Husk at KI skal brukes som hjelpemiddel, ikke som erstatning for egen tenkning og kreativitet. Du skal selv kunne forklare og forsvare løsningen din.

7 Viktige sjekkpunkter

Når dere løser oppgaven, bør dere passe på følgende:

Kompilering/bygging og prosjektstruktur:

NB! Maven er valgfritt i Trondheim

- Er prosjektet et Maven-prosjekt/prosjekt med en ryddig og riktig katalogstruktur?
- Kan prosjektet kompilere uten feil (Maven compile)?
- Kan samtlige enhets-tester i prosjektet kjøres uten at de feiler (Maven test)?
- NB! Prosjektet SKAL kompilere ved å bruke Maven!

Versjonskontroll med git

- Er prosjektet underlagt versjonskontroll med sentralt repo (på GitHub, git.ntnu.no)?
- Sjekkes det inn (commit) jevnlig?
- Beskriver commit-meldingene endringene på en kort og konsis måte?
- Benyttes grener («branches») på en hensiktsmessig måte?
- Benyttes **tags** på riktig måte i forhold til versjonen i pom.xml og på applikasjonen?

Enhetstester

- Har enhetstestene beskrivende navn som dokumenterer hva testene gjør?
- Er testene godt nok dokumentert til at det klart går frem hva som testes, og hva forventet resultat av testene er?
- Følger de mønstret Arrange-Act-Assert (AAA)?
- Tas det hensyn til både positive og negative tilfeller?
- Er testdekningen fornuftig? (Enkelte klasser egner seg ikke for enhetstester, som f.eks. klassen som representerer UI).

Kodekvalitet

- Er koden godt dokumentert i henhold til JavaDoc-standard? (NB! samtlige metoder som er public **skal** dokumenteres, også enkle metoder som get()- og set()-metoder).
- Er koden robust (unntakshåndtering, validering med mer)?
- Har variabler, metoder og klasser beskrivende navn?
- Er klassene gruppert i en logisk pakkestruktur?
- Har koden god struktur, med **løse koblinger** og **høy kohesjon**?
- Benyttes funksjonell programmering (lambda og streams) på en hensiktsmessig måte der det er naturlig?
- Benyttes verktøy som for eksempel CheckStyle for sjekk av kodestil og doumentasjon?

7.1 Nyttige referanser

- Kodekvalitet og beste praksis:
  https://www.ntnu.no/wiki/display/idatx1001/Kodekvalitet+og+beste+praksis
- Kodestilsjekk med CheckStyle og SonarLint for IntelliJ:
  https://www.ntnu.no/wiki/display/idatx1001/CheckStyle+and+SonarLint+for+IntelliJ

- Kildekodekontroll med GitHub/git.ntnu.no:
  https://www.ntnu.no/wiki/pages/viewpage.action?pageId=235998273
- JUnit-testing og Maven:
  https://www.ntnu.no/wiki/pages/viewpage.action?pageId=240747880

A Vedlegg

Her finner dere lenke til nyttige ressurser, samt tips og triks.

A.1 Nyttige verktøy

For en oversikt over nyttige verktøy til bruk ved utvikling av programvare, se vår Wiki:

https://www.ntnu.no/wiki/display/idatx1001. (Se under menyen «Verktøy og IDE-er»).

A.1.1 UML-modellering

PlantUML Finnes som plugin til IntellJ. Gratis/Open Source. Støtter samtlige UML-diagramtyper med mer. Se: https://www.ntnu.no/wiki/display/idatx1001/Plugins+for+IntelliJ+IDE

Hvor hører PlantUML-filene hjemme i Maven-prosjekt strukturen? Forslag:

- **src/doc/plantuml/:** legg PlantUML-filene her (de som ender på "***.puml***").
- **docs/uml/:** legg de genererte PNG-diagramfilene her.

B KI-deklarasjon

En KI-deklarasjon som *skal* leveres inn utfylt, er vedlagt på neste side. Last ned Word-malen og fyll den ut, og last opp generert PDF. Du kan også benytte den forenklede deklarasjonen (Se siden Kunstig intelligens i studentoppgaver under overskiften **KI-deklarasjonsskjema**).

Deklarasjon om KI-hjelpemidler

Har det i utarbeidingen av denne rapporten blitt anvendt KI-baserte hjelpemidler?

☐ Nei

☐ Ja

Hvis *ja*: spesifiser type av verktøy og bruksområde under.

Tekst

☐ **Stavekontroll**. Er deler av teksten kontrollert av:
Grammarly, Ginger, Grammarbot, LanguageTool, ProWritingAid, Sapling, Trinka.ai eller lignende verktøy?

☐ **Tekstgenerering**. Er deler av teksten generert av:
ChatGPT, GrammarlyGO, Copy.AI, WordAi, WriteSonic, Jasper, Simplified, Rytr eller lignende verktøy?

☐ **Skriveassistanse**. Er en eller flere av ideene eller fremgangsmåtene i oppgaven foreslått av:
ChatGPT, Google Bard, Bing chat, YouChat eller lignende verktøy?

Hvis ja til anvendelse av et tekstverktøy - spesifiser bruken her:

Kode og algoritmer

☐ **Programmeringsassistanse**. Er deler av koden/algoritmene som i) fremtrer direkte i rapporten eller ii) har blitt anvendt for produksjon av resultater slik som figurer, tabeller eller tallverdier blitt generert av: *GitHub Copilot, CodeGPT, Google Codey/Studio Bot, Replit Ghostwriter, Amazon CodeWhisperer, GPT Engineer, ChatGPT, Google Bard* eller lignende verktøy?

Hvis *ja t*il anvendelse av et programmeringsverktøy - spesifiser bruken her:

Bilder og figurer

☐ **Bildegenerering**. Er ett eller flere av bildene/figurene i rapporten blitt generert av: *Midjourney, Jasper, WriteSonic, Stability AI, Dall-E* eller lignende verktøy?

Hvis ja til anvendelse av et bildeverktøy - spesifiser bruken her:

Andre KI-verktøy

☐ **Andre KI-verktøy.** har andre typer av verktøy blitt anvendt? Hvis ja spesifiser bruken her:

☐ Jeg er kjent med NTNUs regelverk: *Det er ikke tillatt å generere besvarelse ved hjelp av kunstig intelli- gens og levere den helt eller delvis som egen besvarelse.* Jeg har derfor redegjort for all anvendelse av kunstig intelligens enten i) direkte i rapporten eller ii) i dette skjemaet

---

Underskrift/Dato/Sted

NTNU

Deklarasjon

Har det i utarbeidingen/utformingen/tilvirkingen av denne besvarelsen/oppgaven blitt brukt KI-baserte hjelpemidler?

Nei Ja

Hvis nei, signer:

Hvis ja, fyll ut resten av deklarasjonsskjemaet.

Jeg tar fullt ansvar for teksten som denne deklarasjonen gjelder for. Jeg har kryssjekket kildene og dataene som kildene er basert på, og jeg forstår argumentene som presenteres. Jeg har ikke brukt kilder som jeg ikke anerkjenner som troverdige.

I arbeidet har jeg tatt i bruk følgende KI-baserte hjelpemidler:

Jeg har brukt KI-baserte hjelpemidler på følgende måte (inntil 200 ord):

Eksempler:

«Basert på en første idéutvikling brukte jeg Copilot til å skissere ulike mulige svar. Deretter brukte jeg Google Scholar for å finne kilder som kunne støtte eller avkrefte disse svarene»

«Jeg brukte ChatGPT for å oppsummere og forklare avsnitt." / "Etter dette oppsummerte jeg resultatene og formulerte tekstavsnitt før jeg spurte ChatGPT om å forbedre tekstflyten»

Jeg er kjent med NTNUs regelverk for bruk av kunstig intelligens. Jeg har redegjort for all bruk av kunstig intelligens enten i) direkte i rapporten eller ii) i dette skjemaet.

Underskrift/Dato/Sted

NTNU

Deklarasjon

Har det blitt brukt KI-baserte hjelpemiddel utforminga/tilverkinga av denne oppgåva??

Nei Ja

Viss nei, signer:

Viss ja, fyll ut resten av deklarasjonsskjemaet.

Eg tar fullt ansvar for teksten som denne deklarasjonen gjeld for. Eg har kryssjekka kjeldene og dei data som kjeldene er basert på, og eg forstår argumenta som blir presentert. Eg har ikkje brukt kjelder som eg ikkje anerkjenner som truverdige.

I arbeidet har eg tatt i bruk følgjande KI-baserte hjelpemiddel:

Eg har brukt KI-baserte hjelpemiddel på følgjande måte (inntil 200 ord):

Døme:

«Basert på ei første idéutvikling brukte eg Copilot for å skissere ulike svaralternativ. Deretter blei Google Scholar brukt for å søkje etter kjelder som ga støtte eller motbevis.»

«Eg brukte ChatGPT for å samanfatte og forklare avsnitt.» / «Etter dette samanfatta eg resultata og formulerte tekstavsnitt, før eg spurte ChatGPT om å forbetre tekstflyten.»

Eg er kjend med NTNU sitt regelverk for kunstig intelligens. Eg har gjort greie for all bruk av kunstig intelligens anten i) direkte i rapporten eller ii) i dette skjemaet.

Underskrift/Dato/Stad